REPORT

Reviewer report: Delgan/loguru#1515: Evaluate every

rotation condition so a skipped one cannot fire spuriously

Run loguru-1515, tested 2026-11-09 12:39 IST by ops/daily_run.py (route source-pair). Proof-of-Change commit

ef12096677e9cf59e8673436d74ec65d53771d7c. Report generated 2026-11-09 15:07 IST by

reports/pr_report.py.

Baseline: full (test comparison + claim probes + coverage gate + claim flagging (differential probes shown,

informational)).

VERIFIED

Recommended action: Approve with notes Confidence: High

The PR's tests behave the same or better than on main, in all 3 runs per side. Every check passed and every

claim cites a log line.

Advice only: Proof-of-Change never blocks a merge and never posts on GitHub.

Summary in plain English

• What it says it does: Evaluate every rotation condition so a skipped one cannot fire spuriously

(Delgan/loguru#1515).

• What we observed: 1 test(s) fail on main and pass on the PR; 148 test(s) ran on main, 148 on the PR; 0 new

test(s) pass, 150 unchanged (3 of them skipped on both sides); probe: 3 unchanged sub-check(s) of an input

where the claim's change was seen, shown as advisory only: c1.ret1 (C1); c1.second_limit_advanced (C1);

c1.second_limit_after_first (C1); probe: claim(s) C1, C2, C3 proven.

• What you should do: You can approve; read the notes under 'What went wrong' first.

Demo helper

These notes are background from the console and the daily check. They do not change the verdict.

If setup fails, the helper retries the install up to 2 times. If a test still can't be verified because something is

missing or fails to import, it looks up the cause, installs the fix on both sides and reruns that step, up to 3 tries.

• Setup check: GitHub token: present.

• Setup check: Nebius token: present.

• Setup check: free memory 9.7 GB (of 15.6 GB).

• Setup check: free disk 92.0 GB.

• Demo helper: git is installed.

• Demo helper: python3 is installed.

• Demo helper: uv is installed.

• Looked up with Tavily: [redacted] results

• Query: loguru python setup install instructions https://github.com/Delgan/loguru site:github.com/Delgan/loguru

• Query: loguru python documentation install https://github.com/Delgan/loguru site:github.com/Delgan/loguru

• Query: loguru python install requirements https://pypi.org/project/loguru/ https://github.com/Delgan/loguru

site:pypi.org

• Source: loguru/pyproject.toml at master · Delgan/loguru · GitHub —

https://github.com/Delgan/loguru/blob/master/pyproject.toml

• Source: lager · PyPI — https://pypi.org/project/lager/

• Source: Python Logging (beans-logging) - PyPI — https://pypi.org/project/beans-logging/

• Source: loguru - PyPI — https://pypi.org/project/loguru/

• Source: loguru/CONTRIBUTING.rst at master · Delgan/loguru · GitHub —

https://github.com/Delgan/loguru/blob/master/CONTRIBUTING.rst

• Source: GitHub - Delgan/loguru: Python logging made (stupidly) simple · GitHub —

https://github.com/delgan/loguru

• Source: Can‘t Import loguru · Issue #116 · Delgan/loguru · GitHub —

https://github.com/Delgan/loguru/issues/116

• Source: Suggestion/request: Documentation/tutorial for beginners · Issue #71 · Delgan/loguru · GitHub —

https://github.com/Delgan/loguru/issues/71

• Setup plan: the same packages on main and pr.

• Setup plan: colorama is noted, installed with the project.

• Setup plan: aiocontextvars is noted, installed with the project.

• Setup plan: win32-setctime is noted, installed with the project.

• Setup plan: install pre-commit==4.6.0 ; python_version>='3.11' (listed in the dev extra) on main and pr.

• Setup plan: install tox==3.27.1 ; python_version<'3.8' (listed in the dev extra) on main and pr.

• Setup plan: install pytest==6.1.2 ; python_version<'3.8' (listed in the dev extra) on main and pr.

• Setup plan: install pytest-cov==2.12.1 ; python_version<'3.8' (listed in the dev extra) on main and pr.

• Setup plan: install pytest-mypy-plugins==4.0.3 ; python_version>='3.12' (listed in the dev extra) on main and pr.

• Setup plan: install freezegun==1.1.0 ; python_version<'3.8' (listed in the dev extra) on main and pr.

• Setup plan: install exceptiongroup==1.1.3 ; python_version>='3.7' and python_version<'3.11' (listed in the dev

extra) on main and pr.

• Setup plan: install mypy==v0.910 ; python_version<'3.6' (listed in the dev extra) on main and pr.

• Setup plan: install Sphinx==8.2.1 ; python_version>='3.11' (listed in the dev extra) on main and pr.

• Setup plan: install sphinx-rtd-theme==3.0.2 ; python_version>='3.11' (listed in the dev extra) on main and pr.

• Setup plan: install myst-parser==4.0.0 ; python_version>='3.11' (listed in the dev extra) on main and pr.

• Setup plan: install build==1.3.0 ; python_version>='3.11' (listed in the dev extra) on main and pr.

• Setup plan: install twine==6.2.0 ; python_version>='3.11' (listed in the dev extra) on main and pr.

• Setup plan: Python 3.12: Python 3.12 already meets >=3.5, <4.0,>=3.5

• Installed pre-commit 4.6.0 from pyproject (listed in the dev extra) on main and pr.

• Installed tox 4.56.1 from pyproject (listed in the dev extra) on main and pr.

• Installed pytest 9.1.1 from pyproject (listed in the dev extra) on main and pr.

• Installed pytest-cov 6.2.1 from pyproject (listed in the dev extra) on main and pr.

• Installed pytest-mypy-plugins 4.0.3 from pyproject (listed in the dev extra) on main and pr.

• Installed freezegun 1.5.0 from pyproject (listed in the dev extra) on main and pr.

• Installed exceptiongroup from pyproject (listed in the dev extra) on main and pr.

• Installed mypy 2.1.0 from pyproject (listed in the dev extra) on main and pr.

• Installed Sphinx 8.2.1 from pyproject (listed in the dev extra) on main and pr.

• Installed sphinx-rtd-theme 3.0.2 from pyproject (listed in the dev extra) on main and pr.

• Installed myst-parser 4.0.0 from pyproject (listed in the dev extra) on main and pr.

• Installed build 1.3.0 from pyproject (listed in the dev extra) on main and pr.

• Installed twine 6.2.0 from pyproject (listed in the dev extra) on main and pr.

Claims vs reality

| # | Claim (source) | Status | What we observed | Log line

| C2 | Existing behavior is kept: tests that pass on main still pass (implied by every PR) | proven | no test that

passes on main fails on the PR (150 unchanged, 3 runs per side) | pytest-pr-1.log:9 «148 passed, 3 skipped in

26.35s»

| P-C1 | A skipped condition is left holding a limit that is now in the past, and it therefore fires on the very next

logged message — producing a second, immediate, spurious rotation. (PR description) | not observed | probe

model-p1, c1.ret1: same on both sides: true | probe-model-p1-pr-1.jsonl:1 «"observation": "c1.ret1"»

| P-C2 | This keeps the documented 'any of which can trigger the rotation' contract for the return value while

restoring the side effect the conditions rely on. (PR description) | proven | probe model-p1, c2.return_value: same

on both sides: true | probe-model-p1-pr-1.jsonl:6 «"observation": "c2.return_value"»

| P-C3 | Fails before the change with assert 3 == 2; passes after. (PR description) | proven | probe model-p1,

c3.rotations_count: value: 2 -> 1 | probe-model-p1-pr-1.jsonl:7 «"observation": "c3.rotations_count"»

Citation check (PASS): 4 citation(s), every one re-read from its log file and matched.

What changed in behavior

| What | Count | Examples

| Tests that actually ran (main / PR, fewest of the runs) | 148 / 148 | | Tests unchanged | 150 | | New tests that pass | 0 | | Tests that pass on main but fail on the PR | 0 | | Tests that are new in the PR and fail | 0 | | Tests that ran on main but are missing on the PR side | 0 | | Tests that changed result in another way (for example passed -> skipped) | 0 | | Tests that gave different results between the 3 runs of the same side (flaky) | 0 | | Tests that fail on main and pass on the PR | 1 |

tests.test_filesink_rotation::test_multiple_rotation_conditions_all_evaluated

| Tests that are new or changed but did not run because an optional module is missing | 0 | | Tests that were re-parametrized (same test, new parameter ids; not counted as removed) | 0 | | Test modules that fail to load on the PR side only (their tests no longer run) | 0 | | Test modules that fail to load on main only (no baseline for their tests) | 0 | | Test modules that fail to load on both sides (their tests did not run) | 0 | | Tests that fail identically on both sides (already broken on main) | 0 | -

Probes

Model-written probes: model provider: nebius (Token Factory, live).

A probe is a small deterministic script run on both sides in the same environments as the tests, 3 alternating runs

per side. It prints JSON observations; plain code (poc/diff.py) compares them. A difference that no claim of the

PR covers needs attention. Every probe was frozen (content sha256) before any PR-side code ran, and checked

against that freeze before each PR-side run. Only a live model probe is blind; fixed probes and recorded model

fixtures are known-answer cases.

model-p1 (model (nvidia/nemotron-3-super-120b-a12b, live), model-probes/model-p1.py, frozen 9e426495dd92

at 2026-11-09T07:12:05.393230+00:00): Test multiple rotation conditions evaluation and side effects.

• P-C1 (PR description): A skipped condition is left holding a limit that is now in the past, and it therefore fires on

the very next logged message — producing a second, immediate, spurious rotation. Expects the covered

observations to differ.

• P-C2 (PR description): This keeps the documented 'any of which can trigger the rotation' contract for the return

value while restoring the side effect the conditions rely on. Expects the covered observations to stay the same.

• P-C3 (PR description): Fails before the change with assert 3 == 2; passes after. Expects the covered

observations to differ.

| Observation | Claim | Result | What the probe saw (main -> PR) | Log lines (main, PR)

| c1.ret1 | P-C1 | no change although the claim says it changes | same on both sides: true |

probe-model-p1-main-1.jsonl:1 «"observation": "c1.ret1"»
probe-model-p1-pr-1.jsonl:1 «"observation":

"c1.ret1"»

| c1.ret2 | P-C1 | as claimed | value: true -> false | probe-model-p1-main-1.jsonl:2 «"observation":

"c1.ret2"»
probe-model-p1-pr-1.jsonl:2 «"observation": "c1.ret2"»

| c1.second_limit_advanced | P-C1 | no change although the claim says it changes | same on both sides: true |

probe-model-p1-main-1.jsonl:3 «"observation": "c1.second_limit_advanced"»
probe-model-p1-pr-1.jsonl:3

«"observation": "c1.second_limit_advanced"»

| c1.second_limit_after_first | P-C1 | no change although the claim says it changes | same on both sides:

"2020-01-01T12:00:00" | probe-model-p1-main-1.jsonl:4 «"observation":

"c1.second_limit_after_first"»
probe-model-p1-pr-1.jsonl:4 «"observation": "c1.second_limit_after_first"»

| c1.total_rotations | P-C1 | as claimed | value: 2 -> 1 | probe-model-p1-main-1.jsonl:5 «"observation":

"c1.total_rotations"»
probe-model-p1-pr-1.jsonl:5 «"observation": "c1.total_rotations"»

| c2.return_value | P-C2 | as claimed | same on both sides: true | probe-model-p1-main-1.jsonl:6 «"observation":

"c2.return_value"»
probe-model-p1-pr-1.jsonl:6 «"observation": "c2.return_value"»

| c3.rotations_count | P-C3 | as claimed | value: 2 -> 1 | probe-model-p1-main-1.jsonl:7 «"observation":

"c3.rotations_count"»
probe-model-p1-pr-1.jsonl:7 «"observation": "c3.rotations_count"»

Citation check (PASS): 14 citation(s), every one re-read from its log file and matched.

Was the changed code exercised?

Coverage: MEASURED by coverage.py on the PR side (28.2 s run).

Coverage gate (python, confidence normal): 5 of 5 changed executable line(s) ran (100%).

| Changed file | Changed executable lines | Of those, ran

| loguru/_file_sink.py | 5 | 5

How this is measured: one extra, untimed run of the PR side's tests under coverage.py. Changed lines come

from comparing the tested main and PR files. Compiled code (C, C++, Cython) is invisible to coverage.py; such a

change only counts as verified when the PR adds or changes a test for that module.

What went right

• Tested 6c91de2c42d3 of the PR against main (source-pair route).

• 3 alternating runs per side: main 3, PR 3 of 3.

• pytest exit codes explained: all runs exited 0.

• Changed code was exercised (coverage gate): 5 of 5 changed executable line(s) ran (100%).

• Probes ran on both sides: 1 probe(s), 3 alternating runs per side, every run printed its observations.

• Single-thread environment: OMP_NUM_THREADS=1, OPENBLAS_NUM_THREADS=1,

MKL_NUM_THREADS=1, NUMEXPR_NUM_THREADS=1, VECLIB_MAXIMUM_THREADS=1,

BLIS_NUM_THREADS=1, PYTHONHASHSEED=0.

• Machine type recorded: local x86_64 8 cpus.

• Nothing posted upstream: GitHub was only read.

• Known-answer replay (fresh re-run): replayed 2026-10-10 06:30:21 IST, harness 1.0.

• Every claim cites a real log line: 4 citation(s), every one re-read from its log file and matched.

• Every probe observation cites its output line: 14 citation(s), every one re-read from its log file and matched.

What went wrong and how to fix it

Nothing went wrong.

Evals and guardrails

| Check | Result | Detail

| 3 alternating runs per side | PASS | main 3, PR 3 of 3

| pytest exit codes explained | PASS | all runs exited 0

| Changed code was exercised (coverage gate) | PASS | 5 of 5 changed executable line(s) ran (100%)

| Probes ran on both sides | PASS | 1 probe(s), 3 alternating runs per side, every run printed its observations

| Single-thread environment | PASS | OMP_NUM_THREADS=1, OPENBLAS_NUM_THREADS=1,

MKL_NUM_THREADS=1, NUMEXPR_NUM_THREADS=1, VECLIB_MAXIMUM_THREADS=1,

BLIS_NUM_THREADS=1, PYTHONHASHSEED=0

| Machine type recorded | PASS | local x86_64 8 cpus

| Nothing posted upstream | PASS | GitHub was only read

| Known-answer replay (fresh re-run) | PASS | replayed 2026-10-10 06:30:21 IST, harness 1.0

| Every claim cites a real log line | PASS | 4 citation(s), every one re-read from its log file and matched

| Every probe observation cites its output line | PASS | 14 citation(s), every one re-read from its log file and

matched

Checklist

• I read the verdict and the plain-English summary.

• Every 'false' or 'unmentioned change' row has a log line I can open.

• Nothing under 'What went wrong' is a run problem (those mean Re-run, not Reject).

• I recorded my decision (see below).

Decision recorded: none yet. To record one, run make decide

REPORT=poc-main/reports/daily/2026-11-09/loguru-1515 DECISION=approve (or reject,

approve_with_notes, re_run).

Rerun command

make check PR=https://github.com/Delgan/loguru/pull/1515

Evidence links

• Pull request Delgan/loguru#1515

• record.json

• pr.diff

• setup-main.log

• setup-pr.log

• git.log

• pytest-main-1.log

• pytest-pr-1.log

• pytest-main-2.log

• pytest-pr-2.log

• pytest-main-3.log

• pytest-pr-3.log

• junit-main-1.xml

• junit-pr-1.xml

• coverage-pytest.log

• coverage-pr.json

• probe-model-p1-main-1.jsonl

• probe-model-p1-pr-1.jsonl

• main run 1 result: pytest-main-1.log:9 «147 passed, 3 skipped in 26.37s»

• pr run 1 result: pytest-pr-1.log:9 «148 passed, 3 skipped in 26.35s»

Glossary

| Term | Meaning

| main | The library as it is today, without the PR.

| PR | Pull request: a proposed change to the library.

| run | One full pass of the selected tests. We do 3 per side, alternating main and PR.

| flaky | A test that gives different results when nothing changed; it is ignored as evidence.

| log line | A line in a pytest-*.log file: the raw output the verdict is based on.

| route | How the two sides were built: nightly+patch, source-build or source-pair (see docs/OPERATIONS.md).

| changed lines covered | How many of the PR's changed, executable Python lines ran during the PR side's tests.

| re-parametrized | A test whose parameter ids changed (renamed cases); the new ids are judged on their own.

rebuilt from cleaned text

Written 2026-11-09 15:07 IST